

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ"

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

Лабораторная работа № 2
по дисциплине "Введение в искусственный интеллект и логическое
программирование"

Факультет: ФПМИ

Группа: ПМИ-32

Студенты: Весёлый Д., Ворончук И., Лыкова М.

Бригада №2

Преподаватели: Целебровская М. Ю.

НОВОСИБИРСК 2026

1. Цель работы

Понять принципы создания, управления и использования внутренних баз данных в Турбо-Прологе, научиться использовать их при проектировании программ.

2. Задание

1. Разработайте индивидуальную программу согласно предлагаемым индивидуальным заданиям таким образом, чтобы все основные факты вашей программы хранились во внутренней базе данных (считывались из файла, обрабатывались программой и снова записывались в файл). При этом введите диалог с пользователем для добавления или удаления фактов (за основу организации диалога возьмите реализацию задания 1).

Индивидуальное задание:

2. Сведения об ученике состоят из его имени и фамилии, пола, названия класса, отметок, полученных учениками в последней четверти.

Спроектировать систему, отвечающую на главный вопрос: возьмет ли школа главный приз? Введены следующие ограничения:

- Школа возьмет главный приз, если в ней будет проведено три олимпиады, конкурс красоты, гонки, не будет беспорядков и число лентяев не более трех.
- Беспорядки происходят, если есть класс, где нет отличников и в нем более четырех человек.
- Гонки проводятся, если в них участвует не менее двух человек.
- Среди участников гонок могут быть только троечники.
- В гонках участвуют и девочки, и мальчики.
- На конкурсе красоты должно быть не менее двух участников и трех судей.
- Судьей не может быть человек, занятый в нескольких олимпиадах.
- Ученик, участвующий в олимпиаде по физике, не может участвовать в конкурсе красоты.
- Только девочки участвуют в конкурсе красоты.
- Ученик участвует в олимпиаде, если он отличник по этому предмету.
- Олимпиада по предмету проводится, если на каждой параллели не менее четырех участников, участвующих в олимпиаде.

3. Тексты программ

domains

```
name = symbol
subject = symbol
mark = integer
class_num = integer
gender = symbol
namelist = name*
subjectlist = subject*
classlist = class_num*
```

database

```
student(name, name, gender, class_num)
grade(name, subject, mark)
participation_race(name)
participation_beauty(name)
participation_olymp(name, subject, class_num)
judge(name)
temp_name(name)
temp_class(class_num)
temp_subject(subject)
```

predicates

```
nondeterm menu
nondeterm check_menu(symbol)
nondeterm add_student
nondeterm add_grade
nondeterm check_prize
nondeterm win_prize
nondeterm has_disorders
nondeterm disorder_in_class(class_num)
nondeterm count_lazy(integer)
nondeterm is_lazy(name)
nondeterm olympiads_held(integer)
nondeterm olympiad_held(subject)
nondeterm races_held
nondeterm beauty_held
nondeterm count_class_students(class_num, integer)
nondeterm count_excellent_in_class(class_num, integer)
nondeterm save_data
nondeterm load_data
nondeterm member_name(name, namelist)
```

```

nondeterm length_namelist(namelist, integer)
nondeterm length_subjectlist(subjectlist, integer)
nondeterm length_classlist(classlist, integer)
nondeterm clear_temp_data
nondeterm collect_names(namelist)
nondeterm collect_classes(classlist)
nondeterm collect_subjects(subjectlist)
nondeterm check_all_classes(classlist, subject)
nondeterm count_olymp_in_class(class_num, subject, integer)
nondeterm check_race_members(namelist)
nondeterm check_all_race_grades(namelist)
nondeterm check_beauty_members(namelist)
nondeterm check_all_female(namelist)
nondeterm check_no_physics(namelist)
nondeterm count_judges(integer)
nondeterm check_all_judges
nondeterm count_judge_olymps(name, integer)
nondeterm findall_temp(namelist)
nondeterm findall_temp_class(classlist)
nondeterm findall_temp_subject(subjectlist)

```

clauses

menu:-

```

    write("\n=== SCHOOL PRIZE SYSTEM ==="),
    write("\n1. Load database"),
    write("\n2. Add student"),
    write("\n3. Add grade"),
    write("\n4. Check Prize"),
    write("\n5. Save and Exit"),
    write("\nSelect: "),
    readln(MenuStr),
    check_menu(MenuStr).

```

```

check_menu("1"):- load_data, menu.
check_menu("2"):- add_student, menu.
check_menu("3"):- add_grade, menu.
check_menu("4"):- check_prize, menu.
check_menu("5"):- save_data, write("\nBye!"), !.
check_menu(_):- write("\nInvalid"), menu.

```

add_student:-

```

    write("\nName: "), readln(Name),

```

```

write("Surname: "), readln(Surname),
write("Gender (male/female): "), readln(Gender),
write("Class: "), readint(Class),
assertz(student(Name, Surname, Gender, Class)),
write("Added.").

add_grade:-
    write("\nStudent Name: "), readln(Name),
    write("Subject: "), readln(Subject),
    write("Mark (2-5): "), readint(Mark),
    assertz(grade(Name, Subject, Mark)),
    write("Added.").

check_prize:-
    write("\n--- Analysis ---"),
    win_prize,
    write("\n*** SCHOOL WINS! ***"), !.
check_prize:-
    write("\n--- School loses ---").

win_prize:-
    olympiads_held(Count0),
    Count0 >= 3,
    write("\n[OK] Olympiads: "), write(Count0),
    beauty_held,
    write("\n[OK] Beauty contest"),
    races_held,
    write("\n[OK] Races"),
    not(has_disorders),
    write("\n[OK] No disorders"),
    count_lazy(CountL),
    CountL <= 3,
    write("\n[OK] Lazy: "), write(CountL).

is_lazy(Name):- grade(Name, _, 2).

count_lazy(Count):-
    clear_temp_data,
    is_lazy(N), assertz(temp_name(N)), fail;
    collect_names(List), length_namelist(List, Count).

disorder_in_class(Class):-

```

```

    student(_, _, _, Class),          % This line "binds" Class
to an existing class number
    count_class_students(Class, CountS),
    CountS > 4,
    count_excellent_in_class(Class, CountE),
    CountE = 0.

has_disorders:- disorder_in_class(_).

count_class_students(Class, Count):-
    clear_temp_data,
    student(S, _, _, Class), assertz(temp_name(S)), fail;
    collect_names(List), length_namelist(List, Count).

count_excellent_in_class(Class, Count):-
    clear_temp_data,
    student(Name, _, _, Class), grade(Name, _, 5),
assertz(temp_name(Name)), fail;
    collect_names(List), length_namelist(List, Count).

olympiad_held(Subject):-
    clear_temp_data,
    participation_olymp(_, Subject, C),
assertz(temp_class(C)), fail;
    collect_classes(Classes),
    check_all_classes(Classes, Subject).

check_all_classes([], _).
check_all_classes([C|Rest], Subject):-
    count_olymp_in_class(C, Subject, Count),
    Count >= 4,
    check_all_classes(Rest, Subject).

count_olymp_in_class(Class, Subject, Count):-
    clear_temp_data,
    participation_olymp(S, Subject, Class),
assertz(temp_name(S)), fail;
    collect_names(List), length_namelist(List, Count).

olympiads_held(Count):-
    clear_temp_data,
    participation_olymp(_, Subj, _),

```

```

olympiad_held(Subj),
assertz(temp_subject(Subj)), fail;
collect_subjects(List), length_subjectlist(List, Count).

races_held:-
    clear_temp_data,
    participation_race(P), assertz(temp_name(P)), fail;
    collect_names(Racers),
    length_namelist(Racers, Count),
    Count >= 2,
    check_race_members(Racers).

check_race_members(Racers):-
    member_name(Male, Racers), student(Male, _, male, _),
    member_name(Fem, Racers), student(Fem, _, female, _),
    check_all_race_grades(Racers).

check_all_race_grades([]).
check_all_race_grades([P|Rest]):-
    grade(P, _, 3),
    check_all_race_grades(Rest).

beauty_held:-
    clear_temp_data,
    participation_beauty(P), assertz(temp_name(P)), fail;
    collect_names(Beauties),
    length_namelist(Beauties, CountB),
    CountB >= 2,
    check_beauty_members(Beauties).

check_beauty_members(Beauties):-
    check_all_female(Beauties),
    check_no_physics(Beauties),
    count_judges(CountJ),
    CountJ = 3,
    check_all_judges.

check_all_female([]).
check_all_female([P|Rest]):-
    student(P, _, female, _),
    check_all_female(Rest).

```

```

check_no_physics([]).
check_no_physics([P|Rest]):-
    not(participation_olymp(P, physics, _)),
    check_no_physics(Rest).
count_judges(Count):-
    clear_temp_data,
    judge(J), assertz(temp_name(J)), fail;
    collect_names(List), length_namelist(List, Count).
check_all_judges:-
    judge(J),
    count_judge_olymps(J, Count),
    Count <= 1,
    fail;
    true.
count_judge_olymps(Judge, Count):-
    clear_temp_data,
    participation_olymp(Judge, _, _),
assertz(temp_name(Judge)), fail;
    collect_names(List), length_namelist(List, Count).
load_data:-
    consult("school", dbasedom),
    write("\nLoaded.").
save_data:-
    save("school", dbasedom).
clear_temp_data:-
    retract(temp_name(_)), fail.
clear_temp_data:-
    retract(temp_class(_)), fail.
clear_temp_data:-
    retract(temp_subject(_)), fail.
clear_temp_data.

collect_names(List):-
    findall_temp(List).

findall_temp([]):-
    not(temp_name(_)).
findall_temp([H|T]):-
    temp_name(H),
    retract(temp_name(H)),
    findall_temp(T).

```



```

collect_classes(List):-
    findall_temp_class(List).

findall_temp_class([]):-
    not(temp_class(_)).
findall_temp_class([H|T]):-
    temp_class(H),
    retract(temp_class(H)),
    findall_temp_class(T).
collect_subjects(List):-
    findall_temp_subject(List).
findall_temp_subject([]):-
    not(temp_subject(_)).
findall_temp_subject([H|T]):-
    temp_subject(H),
    retract(temp_subject(H)),
    findall_temp_subject(T).
member_name(X, [X|_]).
member_name(X, [_|T]):- member_name(X, T).
length_namelist([], 0).
length_namelist([_|T], N):- length_namelist(T, N1), N = N1 +
1.
length_subjectlist([], 0).
length_subjectlist([_|T], N):- length_subjectlist(T, N1), N =
N1 + 1.
length_classlist([], 0).
length_classlist([_|T], N):- length_classlist(T, N1), N = N1 +
1.
goal
menu.

```

Внутренняя база данных:

```

student("Ivan","Ivanov","male",9)
student("Petr","Petrov","male",9)
student("Alex","Smirnov","male",9)
student("Max","Volkov","male",9)
student("Dmitry","Popov","male",10)
student("Sergey","Sokolov","male",10)
student("Roman","Lebedev","male",10)
student("Victor","Kozlov","male",10)
student("Anna","Sidorova","female",9)
student("Oleg","Morozov","male",9)

```

```

student("Elena","Novikova","female",10)
student("Maria","Zaitseva","female",10)
student("Gleb","Ilyin","male",10)
student("Anton","Gusev","male",10)
student("Boris","Titov","male",10)
grade("Ivan","math",5)
grade("Petr","math",5)
grade("Alex","math",5)
grade("Max","math",5)
grade("Dmitry","math",5)
grade("Sergey","math",5)
grade("Roman","math",5)
grade("Victor","math",5)
grade("Ivan","physics",5)
grade("Petr","physics",5)
grade("Alex","physics",5)
grade("Max","physics",5)
grade("Dmitry","physics",5)
grade("Sergey","physics",5)
grade("Roman","physics",5)
grade("Victor","physics",5)
grade("Ivan","informatics",5)
grade("Petr","informatics",5)
grade("Alex","informatics",5)
grade("Max","informatics",5)
grade("Dmitry","informatics",5)
grade("Sergey","informatics",5)
grade("Roman","informatics",5)
grade("Victor","informatics",5)
grade("Anna","history",3)
grade("Oleg","history",3)
participation_race("Anna")
participation_race("Oleg")
participation_beauty("Elena")
participation_beauty("Maria")
participation_olymp("Ivan","math",9)
participation_olymp("Petr","math",9)
participation_olymp("Alex","math",9)
participation_olymp("Max","math",9)
participation_olymp("Dmitry","math",10)
participation_olymp("Sergey","math",10)
participation_olymp("Roman","math",10)
participation_olymp("Victor","math",10)
participation_olymp("Ivan","physics",9)

```

```

participation_olymp("Petr","physics",9)
participation_olymp("Alex","physics",9)
participation_olymp("Max","physics",9)
participation_olymp("Dmitry","physics",10)
participation_olymp("Sergey","physics",10)
participation_olymp("Roman","physics",10)
participation_olymp("Victor","physics",10)
participation_olymp("Ivan","informatics",9)
participation_olymp("Petr","informatics",9)
participation_olymp("Alex","informatics",9)
participation_olymp("Max","informatics",9)
participation_olymp("Dmitry","informatics",10)
participation_olymp("Sergey","informatics",10)
participation_olymp("Roman","informatics",10)
participation_olymp("Victor","informatics",10)
judge("Gleb")
judge("Anton")
judge("Boris")

```

Тесты.

Базовая проверка победы:

1. Загрузить базу (пункт 1 меню).
2. Выбрать «Check Prize» (пункт 4).

Ожидаемый результат: система выводит [OK] по всем пунктам и сообщение *** SCHOOL WINS! ***

Результат выполнения программы:

```

=== SCHOOL PRIZE SYSTEM ===
1. Load database
2. Add student
3. Add grade
4. Check Prize
5. Save and Exit
Select: 1

Loaded.
=== SCHOOL PRIZE SYSTEM ===
1. Load database
2. Add student
3. Add grade
4. Check Prize
5. Save and Exit
Select: 4

— Analysis —
[OK] Olympiads: 3
[OK] Beauty contest held correctly
[OK] Races held correctly
[OK] No disorders found
[OK] Lazy students: 0
*** SCHOOL WINS THE MAIN PRIZE! ***

```

Проверка сохранения и загрузки:

1. Добавить ученика (пункт 2).
2. Сохранить и выйти (пункт 5).
3. Запустить заново, загрузить базу.

Ожидаемый результат выполнения:

Новый ученик отображается в базе, данные не потеряны.

Результат программы:

```
Name: Name1
Surname: Surname1
Gender (male/female): male
Class: 9
Added.
=== SCHOOL PRIZE SYSTEM ===
1. Load database
2. Add student
3. Add grade
4. Check Prize
5. Save and Exit
Select:
```

```
student("Name1","Surname1","male",9)
```

Лимит лентяев.

1. Загрузить базу.
2. Добавить 4 оценки «2» разным ученикам (п.3).
3. Проверить премию (п.4).

Ожидаемый результат: Система сообщает --- School loses --- (превышен лимит лентяей > 3).

Результат программы:

Select: 3

Student Name: Ivan

Subject: Math

Mark [2-5]: 2

Added.

=== SCHOOL PRIZE SYSTEM ===

1. Load database

2. Add student

3. Add grade

4. Check Prize

5. Save and Exit

Select: 4

— Analysis —

— School loses —

Проверка на условие “Участницы красоты и физика”

1. В файле базы добавить факт: participation_olymp("Elena", "physics", 10).

2. Перезагрузить базу.

3. Проверить премию.

Ожидаемый результат: --- School loses --- (нарушено условие check_no_physics).

Результат программы:

=== SCHOOL PRIZE SYSTEM ===

1. Load database

2. Add student

3. Add grade

4. Check Prize

5. Save and Exit

Select: 4

— Analysis —

— School loses —

Неправильный ввод:

=== SCHOOL PRIZE SYSTEM ===

1. Load database

2. Add student

3. Add grade

4. Check Prize

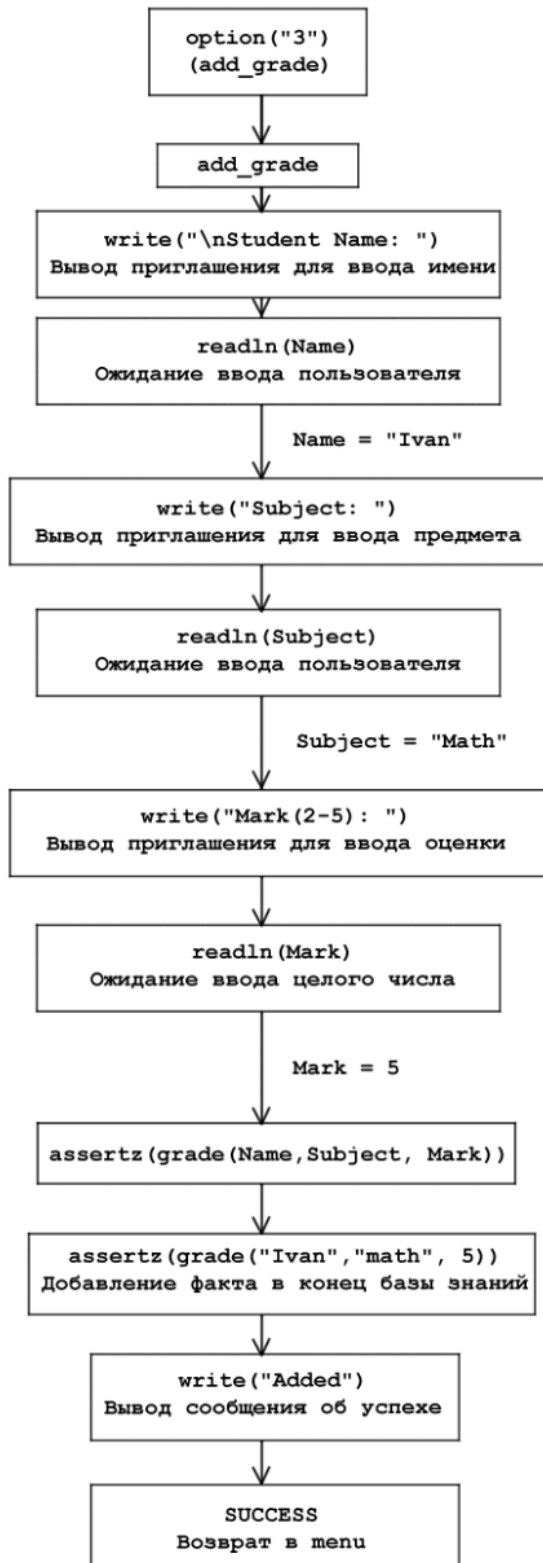
5. Save and Exit

Select: asfad

Invalid

4. Схема поиска решения одной из поставленных целей

Общая схема выполнения запроса добавления оценки:



5. Вывод:

В ходе выполнения лабораторной работы была разработана программа, которая хранит данные во внутренней базе, обрабатывает их согласно заданным правилам и позволяет пользователю управлять информацией через диалоговое меню. Система корректно определяет возможность получения школой главного приза на основе введённых ограничений.